

Memory Match Flutter Game

1. What This Project Is About

So for Exercise 2 I built a memory matching card game using Flutter and Dart. The idea is pretty straightforward, you get a grid of cards all flipped face down, and you tap two at a time trying to find matching animal pairs. When both cards match they stay flipped and get a green border around them. If they don't match they flip back and you try again.

I chose animal images as the card content because it fits the theme of the reference screenshot we were given in the assignment. The whole thing is built using proper Flutter widget composition, `StatefulWidget` for managing game state, and a clean file structure with separate model, screen, and widget files rather than dumping everything into one `main.dart`.

2. Features of the App

- 4x5 grid layout with 20 cards total, which means 10 animal pairs
- All cards start face down with a coloured background and a ? symbol on them
- Tap two cards to flip them and they reveal the animal image underneath
- Matching pairs get a green border and stay flipped permanently
- Non-matching pairs automatically flip back after about 0.85 seconds
- Score counter at the top right that increases by 1 for every successful match
- Win dialog appears when you find all 10 pairs showing your final score
- Restart button at the bottom resets the entire board and shuffles everything again
- `AnimatedSwitcher` handles the smooth card flip transition using `ScaleTransition`
- Responsive grid that automatically adjusts column count based on screen width
- No scrolling needed — all 20 cards fit on screen on any device or window size
- Rapid tap protection so clicking too fast does not break the game logic

3. Project Structure

This is the complete folder structure of the project. I separated everything into different files based on what each part does. The card data model is in its own file, the game logic and screen layout is in its own file, and

the reusable card widget is in a dedicated widgets folder. This is the standard Flutter way of organizing things and it keeps everything much cleaner and easier to work with.

```
space_memory_match/
|
├─ lib/
|   ├─ main.dart           ← App entry point, sets up MaterialApp
|   ├─ game_screen.dart    ← All game logic and screen layout
|   ├─ card_model.dart     ← MemoryCard data model class
|   └─ widgets/
|       └─ memory_card_widget.dart ← Reusable card widget (front and back)
|
├─ assets/
|   └─ images/
|       ├─ animal.png
|       ├─ animal1.png
|       ├─ animal2.png
|       ├─ animals.png
|       ├─ bird.png
|       ├─ bull.png
|       ├─ chick.png
|       ├─ cow.png
|       ├─ elephant.png
|       └─ listening.png
|
├─ test/
|   └─ widget_test.dart
|
├─ pubspec.yaml
├─ pubspec.lock
└─ README.pdf
```

main.dart This is just the entry point of the app. It calls `runApp` and sets up the `MaterialApp` with the theme settings. It imports `game_screen.dart` and points the home to `GameScreen`. I kept it as minimal as possible because the actual work happens in the other files.

card_model.dart This is the data model for a single card. It holds three things — the `imagePath` which tells the widget which animal image to show, `isFlipped` which tracks whether the card is currently face up, and `isMatched` which tracks whether this card has been successfully paired with its partner. Simple Dart class, nothing complicated.

game_screen.dart This is the main file where basically everything happens. It has the list of animal image paths, the list of card back colors, the matching logic, the timer that flips non-matching cards back after 850 milliseconds, the win dialog, and the full Scaffold with AppBar and GridView layout. It uses `StatefulWidget` because the game state is constantly changing as the player taps cards.

widgets/memory_card_widget.dart This is the reusable card widget that gets built for each of the 20 positions in the grid. It receives the card data, the back color, the card index, and an onTap callback from the parent. Inside it decides whether to show the front face which is a white background with the animal image, or the back face which is the colored card with the ? symbol. The AnimatedSwitcher and ScaleTransition animation live here.

4. How to Configure and Run

What you need installed before starting:

- Flutter SDK — download from flutter.dev and make sure it is added to your system PATH
- Android Studio with the Flutter plugin and Dart plugin installed
- An Android Emulator configured in Android Studio Device Manager, or a real Android device with USB debugging enabled in Developer Options

Steps to run:

Step 1 — Verify Flutter is working properly:

```
flutter doctor
```

Flutter, Android toolchain, and Android Studio should all show green checkmarks.

Step 2 — Navigate into the project folder:

```
cd space_memory_match
```

Step 3 — Install all dependencies:

```
flutter pub get
```

Step 4 — Clean the build cache first time:

```
flutter clean
```

```
flutter pub get
```

Step 5 — Run the app:

```
flutter run
```

To run on Chrome browser specifically:

```
flutter run -d chrome
```

To run on Windows desktop:

```
flutter run -d windows
```

Make sure your emulator is already running or your device is connected before doing flutter run. You can also just select the device from the dropdown menu inside Android Studio and click the green play button instead.

5. Technologies Used

- Flutter SDK — cross platform UI framework used to build the entire app
- Dart — programming language the app is written in
- Android Studio — development environment used for coding and testing
- Android Emulator — used for testing on a virtual Android device
- Windows Desktop — used as a secondary testing platform
- Git and GitHub — used for version control and project submission

6. Challenges I Ran Into

The first problem I hit was with the asset images not loading at all. Flutter was not finding any of the image files even though they were clearly sitting in the assets folder. Turned out the `pubspec.yaml` had the assets section commented out by default when the project was created. Once I removed the comment symbols, fixed the indentation, and ran flutter clean followed by flutter pub get, the images started loading correctly.

Another issue was with rapid tapping. If you tapped a third card before the previous non-matching pair had finished flipping back, the game would get into a weird state and sometimes lock up completely. I fixed this by adding a boolean flag called `_isChecking` that prevents any new card taps from being registered while the match check is still being processed. Once the cards either match or flip back, the flag gets set to false again and tapping is re-enabled.

The layout on Windows desktop was also a problem. The cards were so large that you had to scroll down to see the bottom row which looked terrible. I solved this by using `MediaQuery` to get the actual screen height and width, then mathematically calculating exactly how large each card needs to be so that all 20 cards fit within the available space without any scrolling. I also added `NeverScrollableScrollPhysics` to the `GridView` to prevent accidental scrolling even if the calculation is slightly off on edge cases.

Originally all the code was in one single `main.dart` file which became very long and hard to manage. I separated it into four files — `card_model.dart` for the data class, `game_screen.dart` for the main logic and layout, `memory_card_widget.dart` for the reusable card widget, and `main.dart` purely as the entry point. This made the code much more organized and easier to follow.

7. Responsive Design

The app handles different screen sizes in two ways.

First, the column count changes based on screen width:

```
if (screenWidth > 1100) { crossAxisCount = 5; }
```

```
else { crossAxisCount = 4; }
```

Second, the card aspect ratio is calculated dynamically from the actual available screen space so all cards always fit without scrolling:

```
double cardHeight = (availableHeight - (spacing * (rowCount - 1))) / rowCount;
```

```
double cardWidth = (availableWidth - (spacing * (crossAxisCount - 1))) / crossAxisCount;
```

```
double aspectRatio = (cardWidth / cardHeight).clamp(0.5, 2.0);
```

This means on a mobile screen you get 4 columns with appropriately sized cards, on a wider desktop you get 5 columns, and the whole thing recalculates automatically whenever the window is resized. The SafeArea widget also handles device specific insets like the Android navigation bar at the bottom so nothing gets cut off.

8. GitHub Repository

Full project including all Dart files, assets, and this README is available at:

https://github.com/Sukhdeep20/space_memory_match

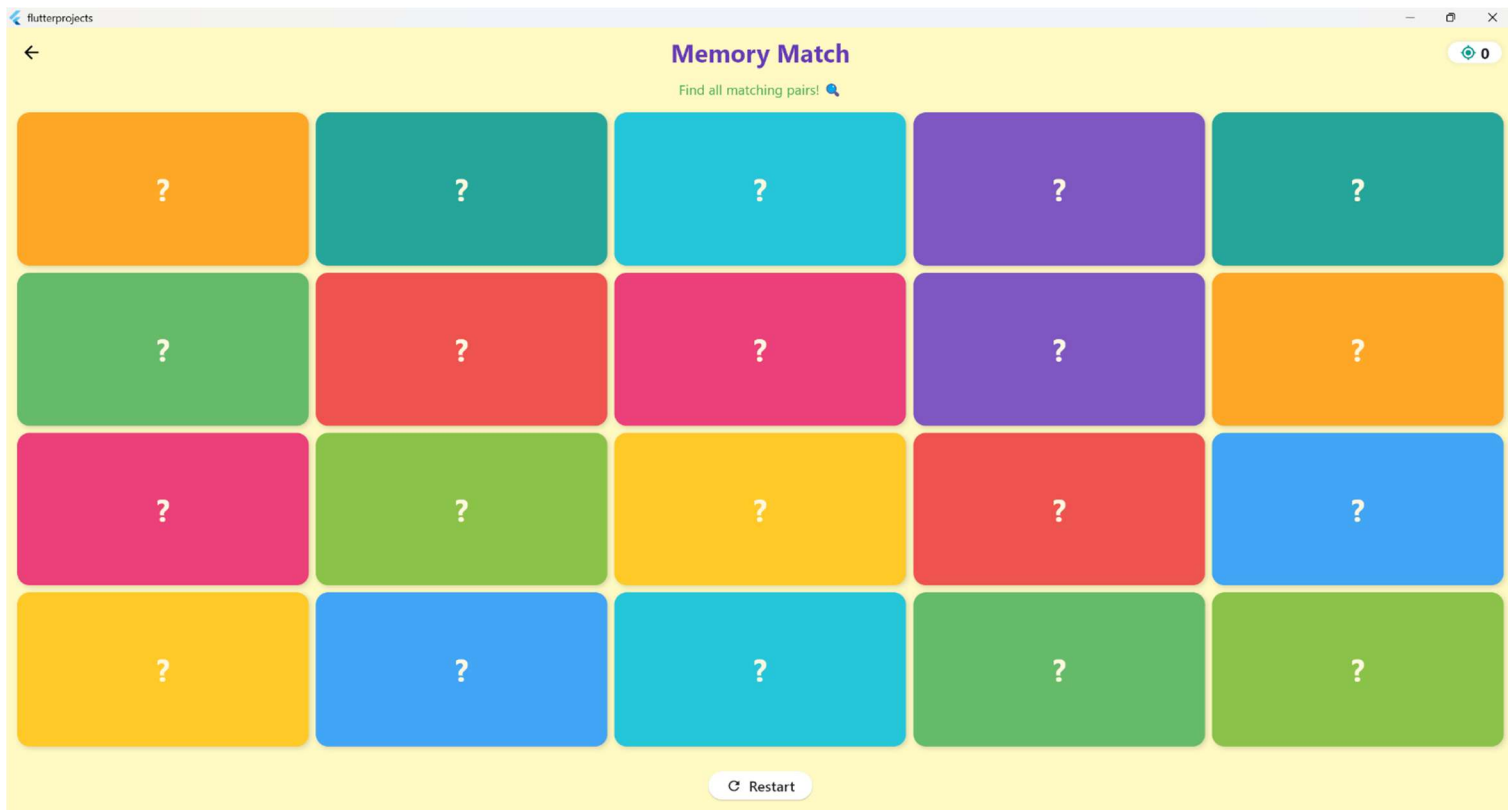
9. What I Would Add With More Time

- Sound effects when a card flips and when a match is found
- A move counter alongside the score so you can see how efficient you were
- A countdown timer to track how fast you finish the board
- Different difficulty levels — easy would be a 3x4 grid, medium 4x5, hard 5x6
- A proper 3D card flip animation instead of the current scale transition
- Local storage for saving high scores between sessions

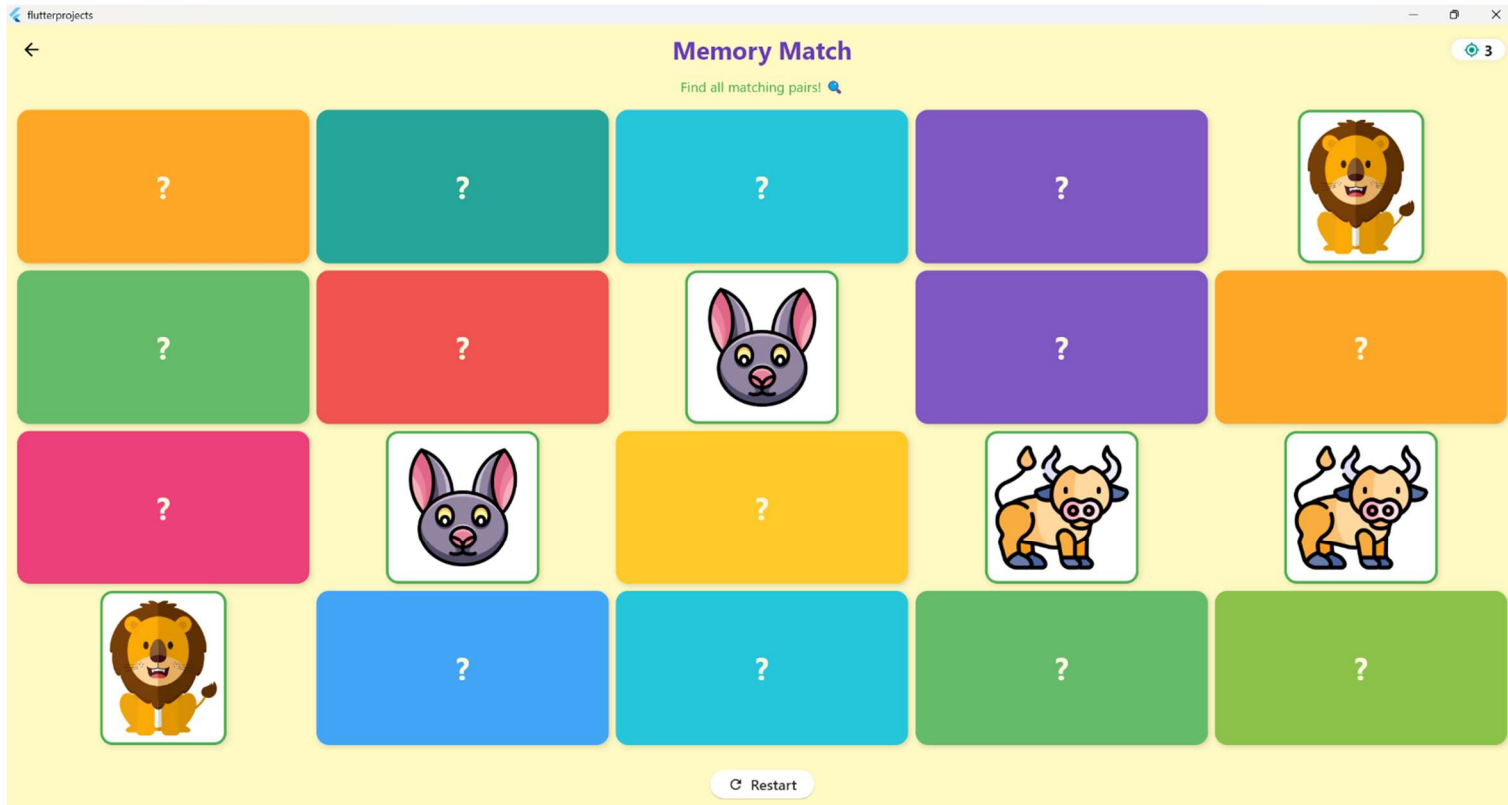
10. Screenshots

Windows App:

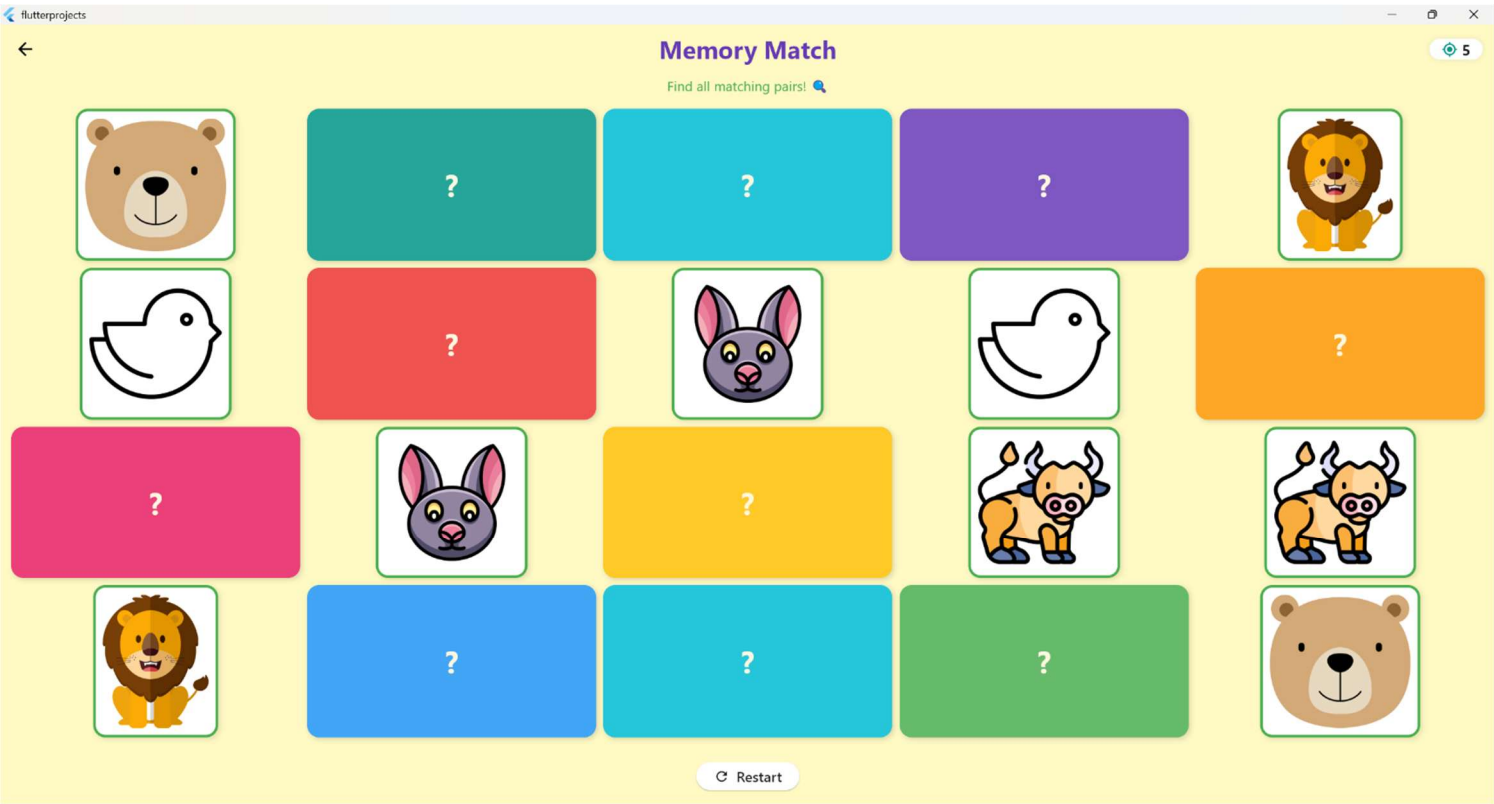
Screenshot 1 - Game start screen, all cards face down



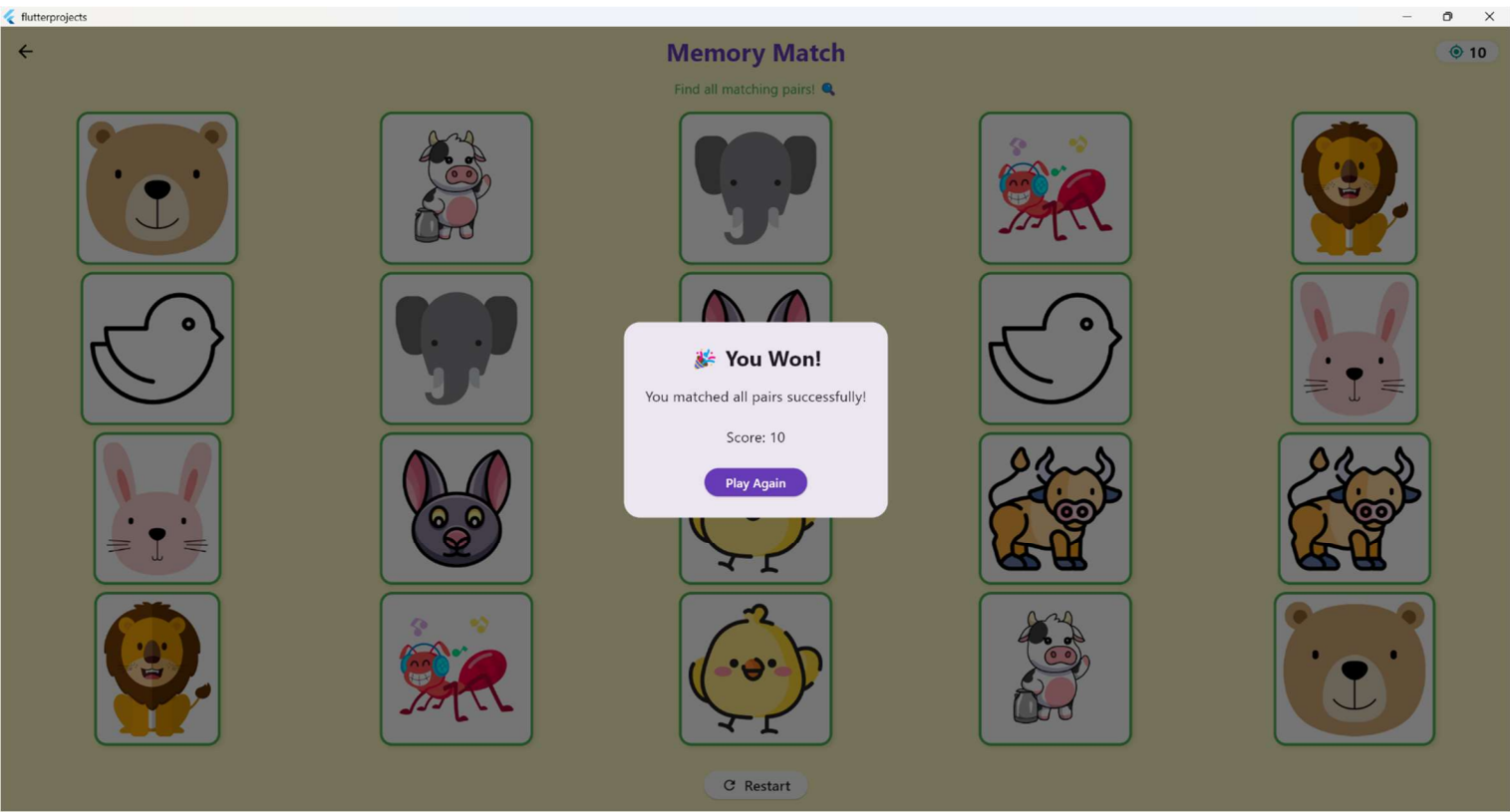
Screenshot 2 - Cards flipped during gameplay



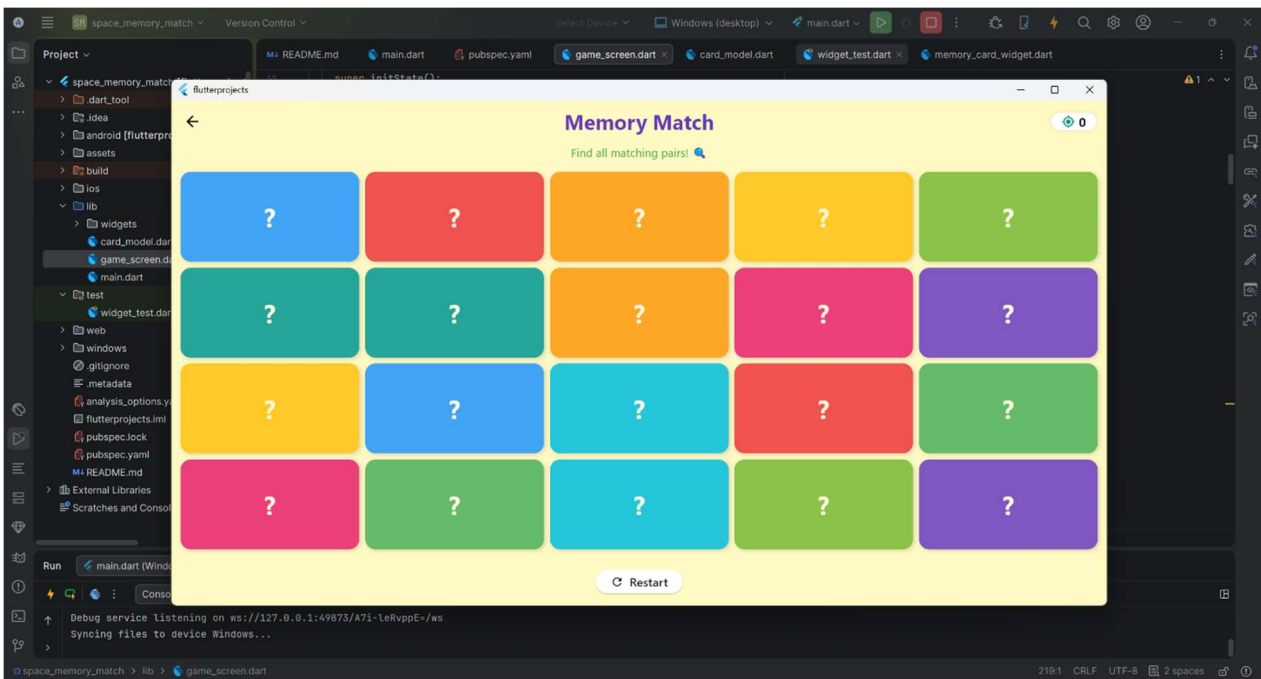
Screenshot 3 - Matched pairs with green borders



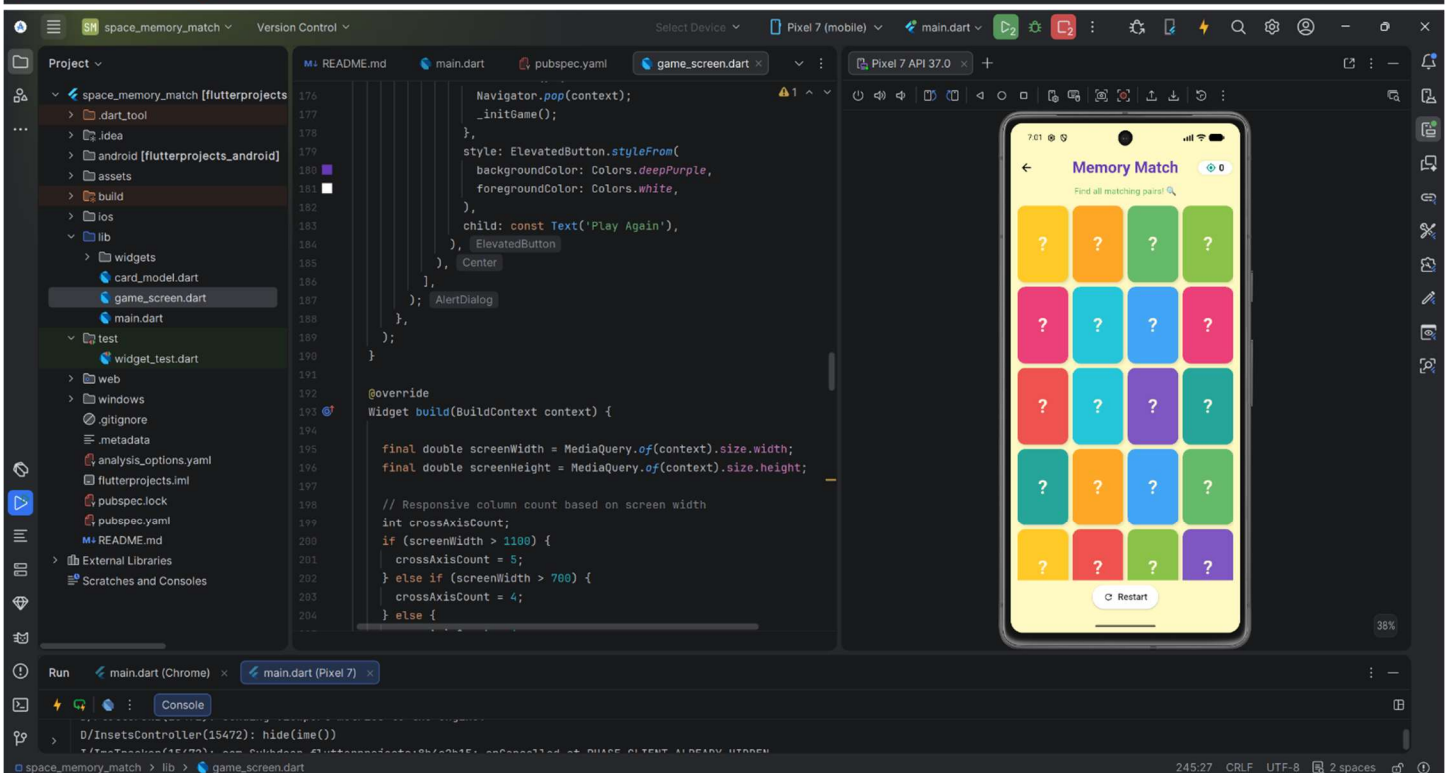
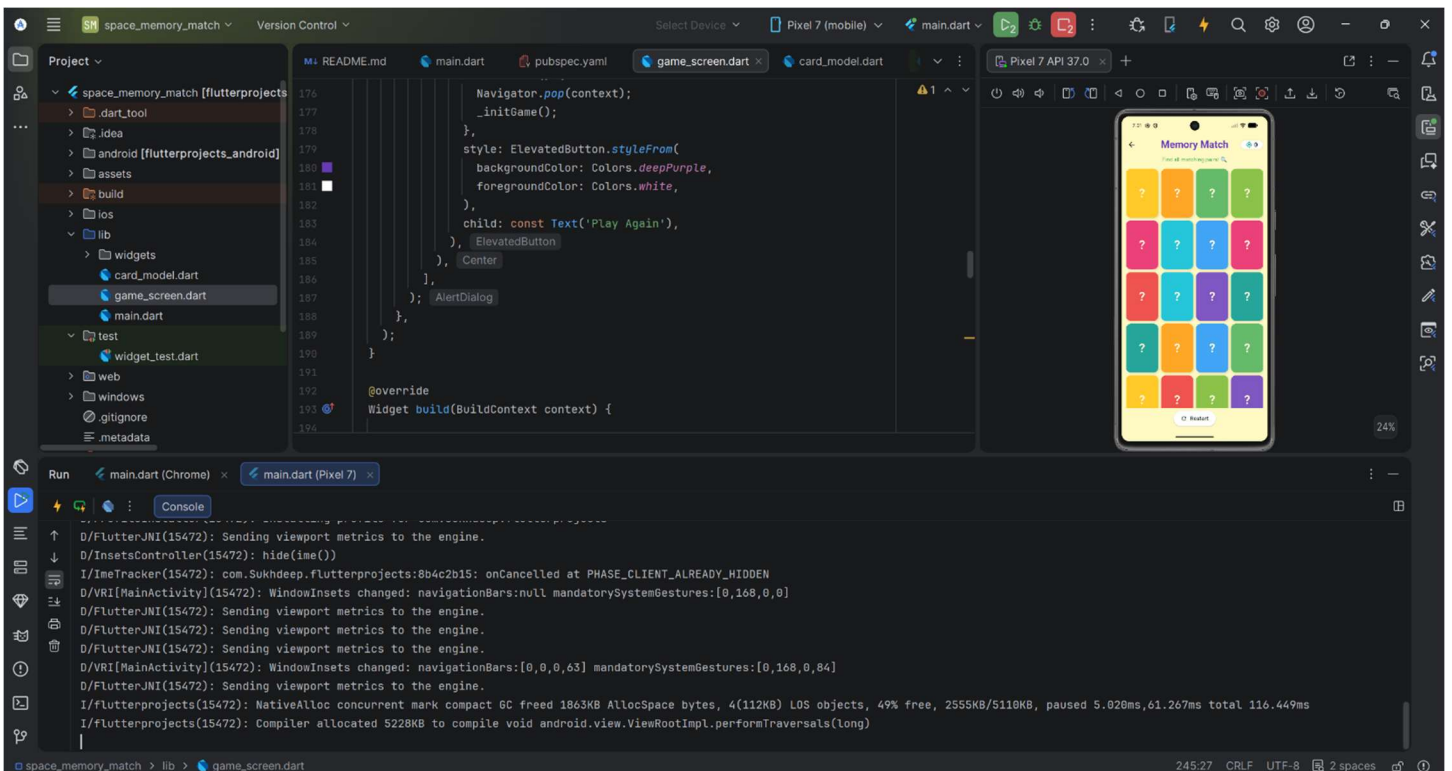
Screenshot 4 - Win dialog after all pairs matched

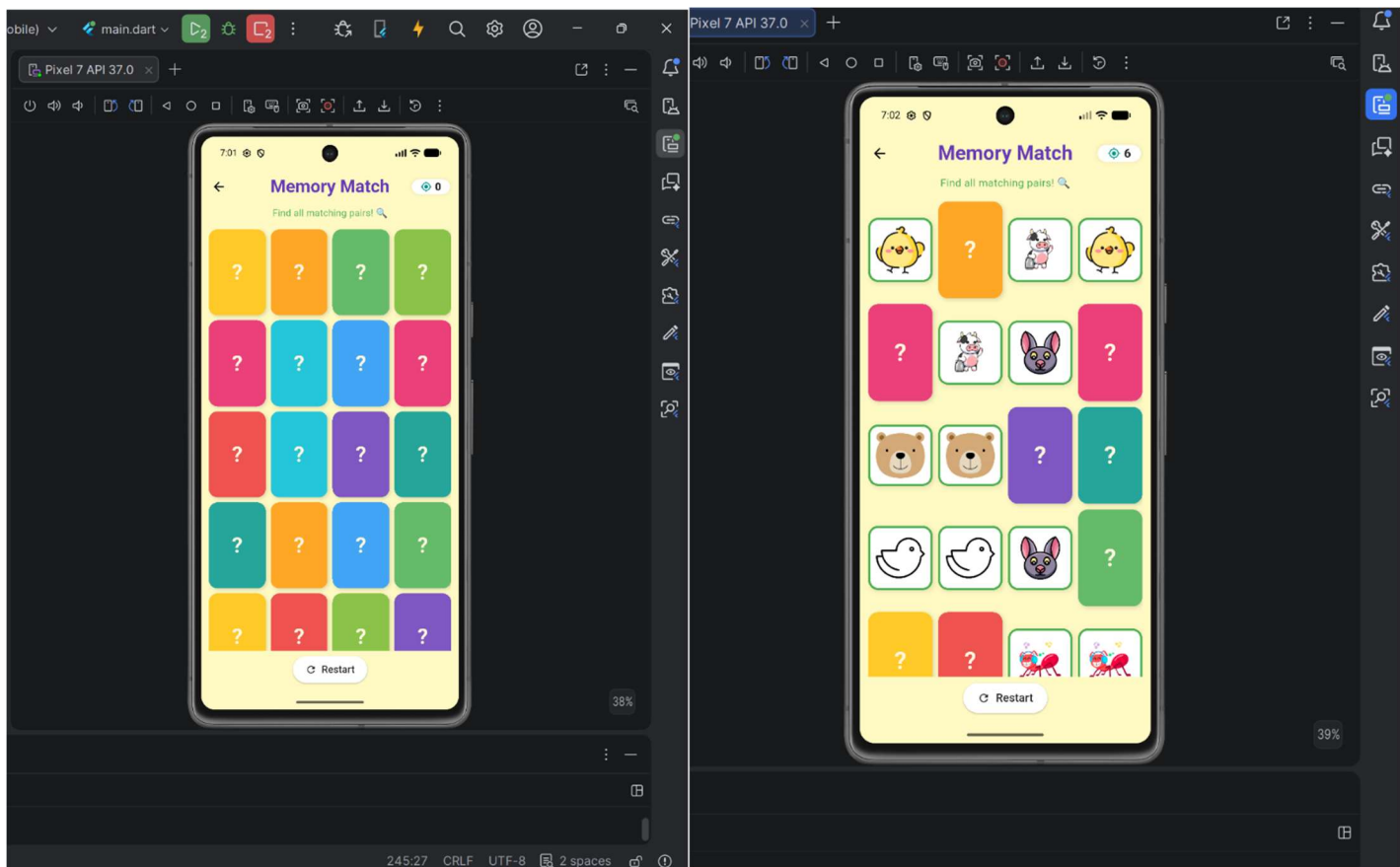


Responsiveness:

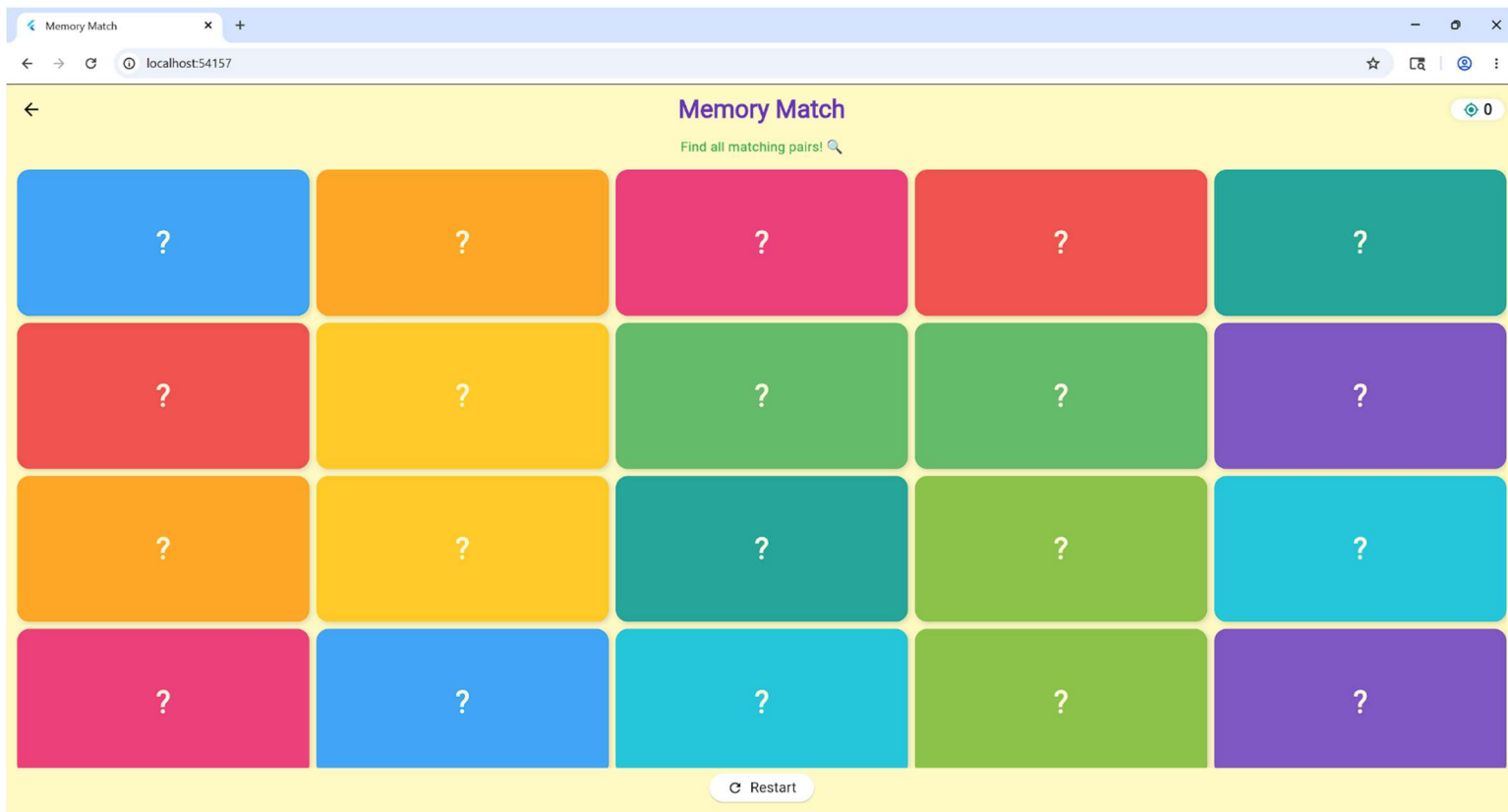


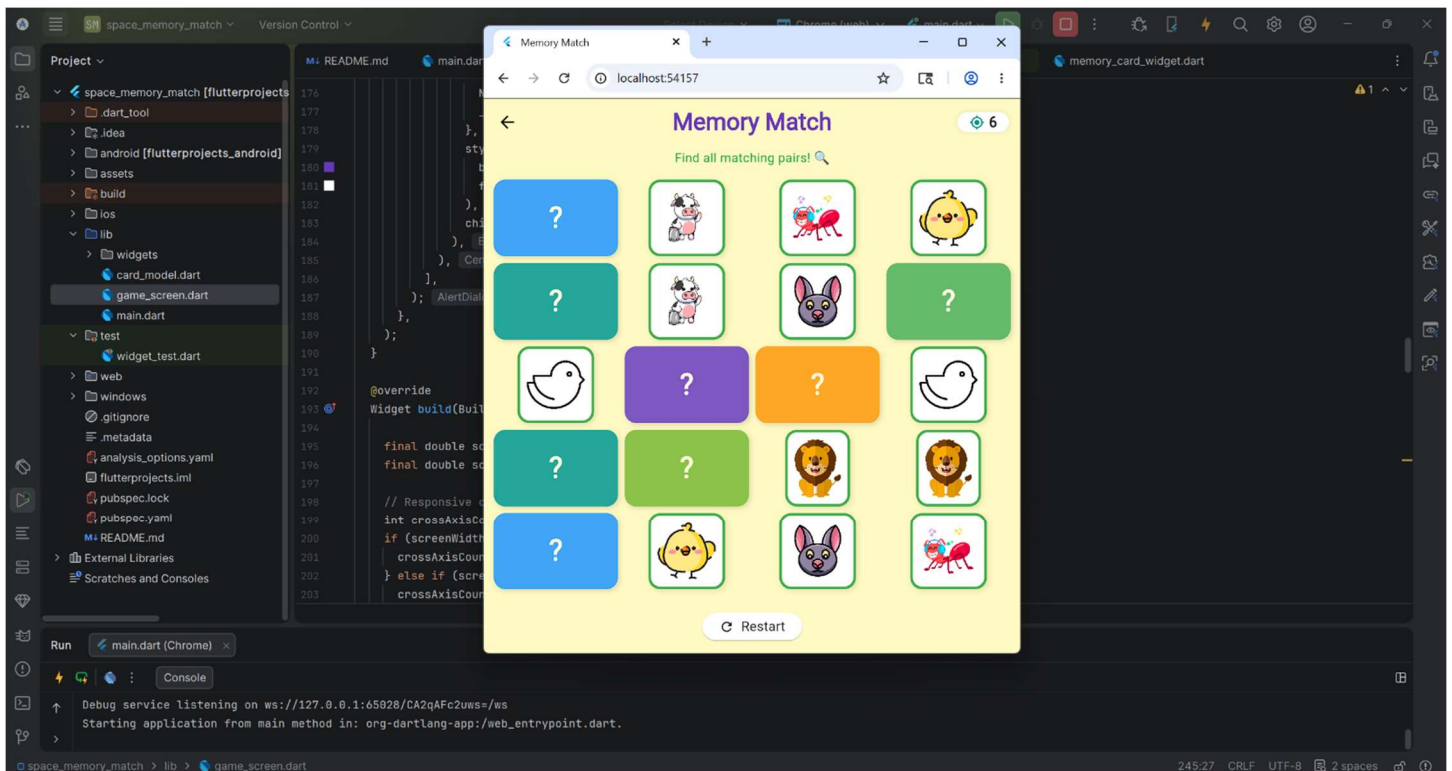
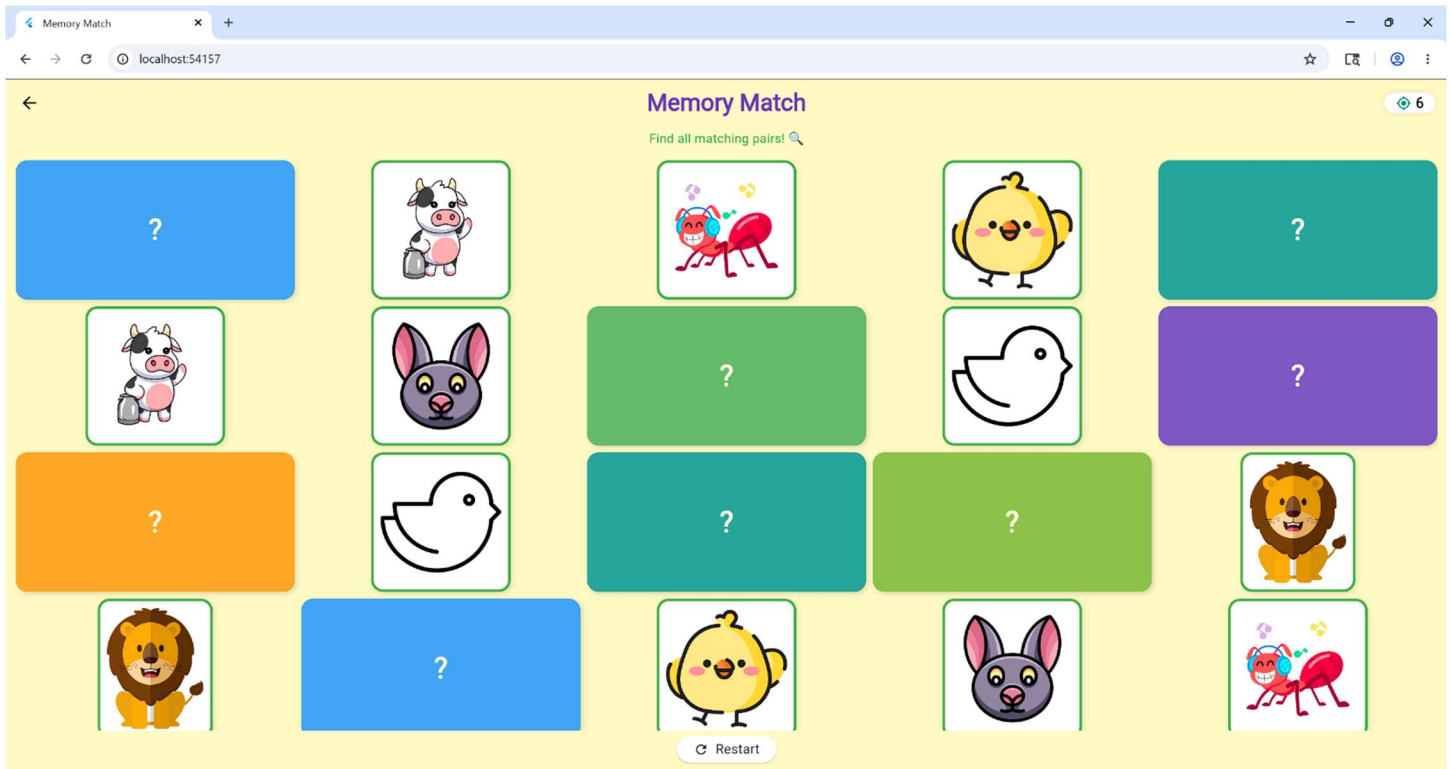
Android Emulator Screenshots:





Chrome Web Screenshots:





Submitted By:

Sukhdeep Singh

1307039

Mobile Programming - CS5450

Lakehead University - Spring 2026